THE 6-AGENT SPECIALIST COUNCIL ARCHITECTURE

Agent 1 (RCA):	Multi-Signal Correlator & AST Inspector	AST parser + BM25/FAISS RAG	**87.8% Grounded Confider**
Agent 2 (Topology):	3D Spatial Mesh Visualizer	BFS Graph Traversal	**Blast Radius Propagation**
Agent 3 (Planner):	Remediation & Cheat Sheet Generator	Unified Diff (difflib) + Runbooks	**Patch & Emergency Rollba**
Agent 4 (Executor):	Sandboxed Action Executor	Atomic Backups + Subprocess Sandbox	**Zero-Regression Automat**
Agent 5 (Guardian):	Safety Policy Governor & Failover	Deterministic Tier Rules + Redundancy	**Authoritative Risk Gateke**
Agent 6 (EVO):	Backend Evolution Engine	Contextual Bandit / Q-Learning	**Proactive Headroom Refa**

MATHEMATICAL FORMULATIONS & THEORETICAL FOUNDATIONS

Rigorous mathematical definitions governing diagnosis, retrieval, blast radius, and reinforcement learning.

w = [0.

Curated

smoothi

ic

{out}}(v

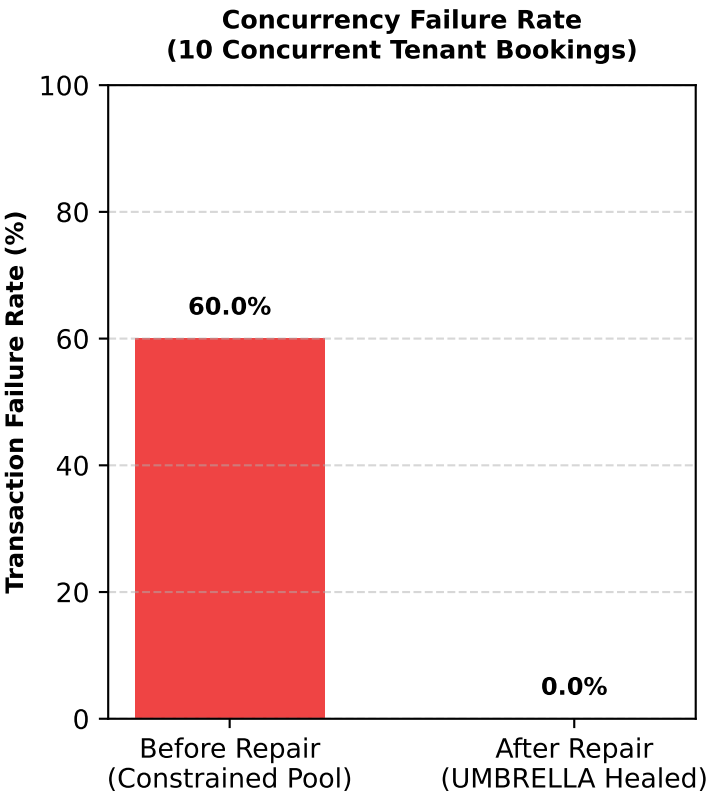
ng

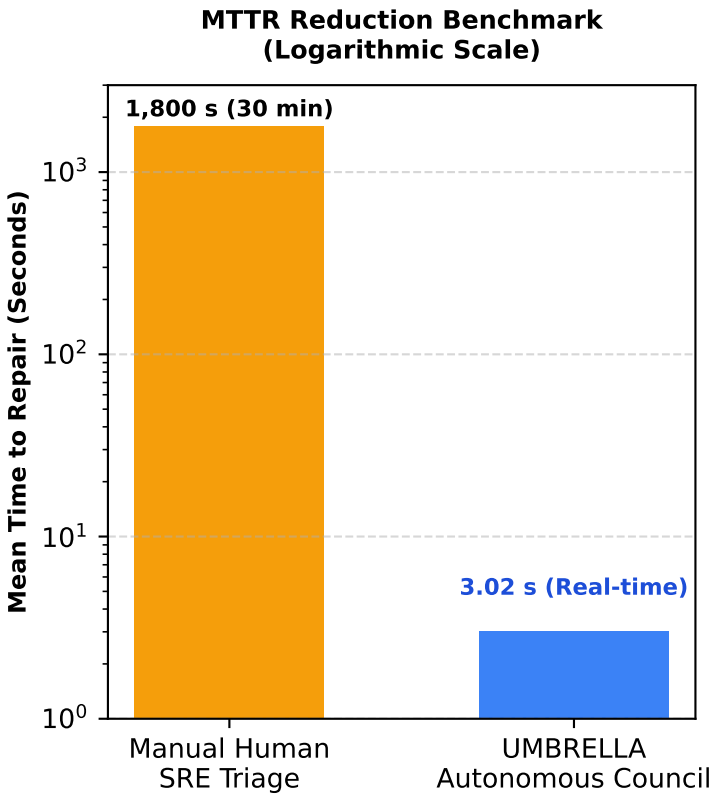
= 0.25

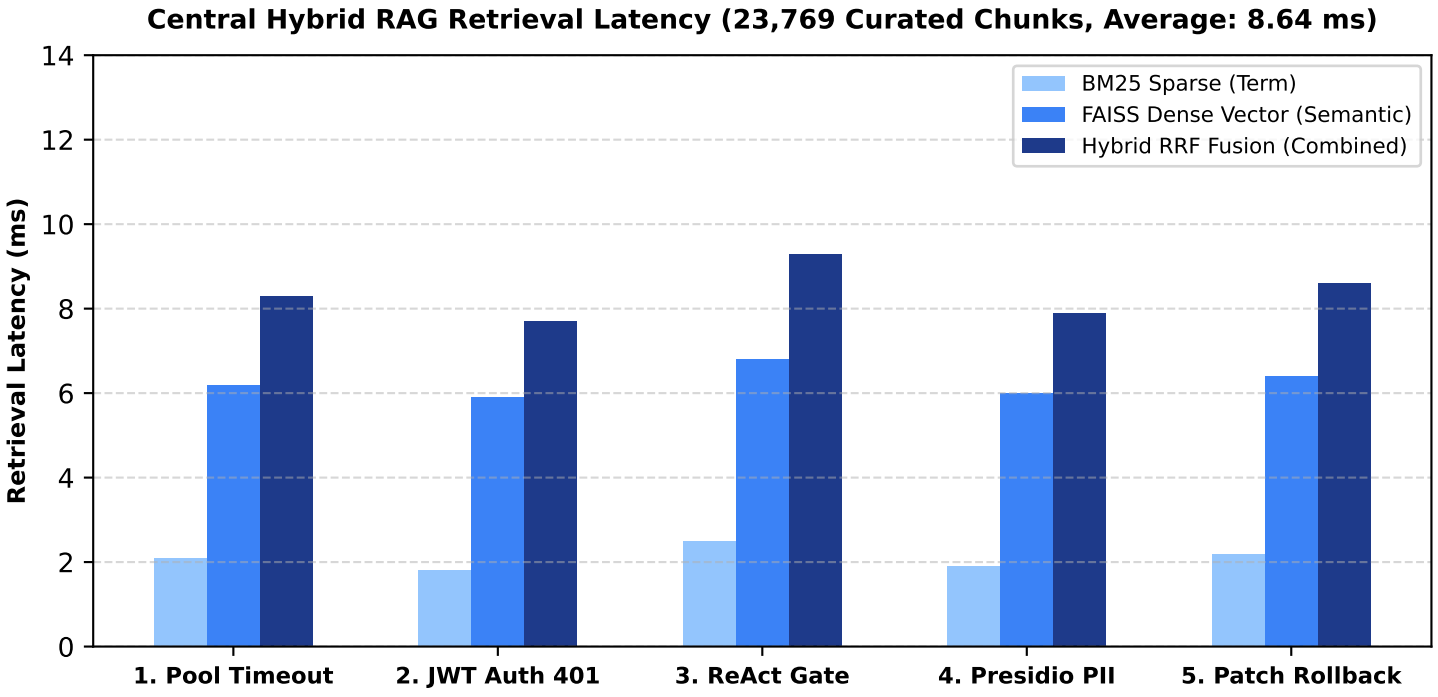
empirical

EMPIRICAL EXPERIMENTAL EVALUATION & BENCHMARKS

Quantitative benchmarks across concurrency stress tests, RAG retrieval latency, and MTTR reduction.







REAL INCIDENT CASE STUDY: AUTONOMOUS CLOSED-LOOP TRAJECTORY

End-to-end execution trace on the Karamstay Room Booking & Rent Payment microservice.



5 concurrent tenants execute room booking transactions against constrained SQLite QueuePool (pool_size=2, timeout=1.0s). Exactly 2 transactions succeed and 3 crash with sqlalchemy.exc.TimeoutError: QueuePool limit of size 2 overflow 0 reached. Saturation hits 250%.



Agent 1 inspects the Python traceback via AST syntax analysis, extracts target_system/demo_app/config.py:10, and queries Central Hybrid RAG. With 4-signal convergence, Agent 1 diagnoses 'Database connection pool starvation' with 87.8% confidence (Critical Impact).



Agent 2 executes bidirectional BFS traversal across the microservice mesh. Identifies that failure in demo_booking_service cascades to API Gateway, Auth Service, and Database (100% blast radius, 6 nodes degraded).



Agent 3 generates a concrete unified diff scaling POOL_SIZE=10 and MAX_OVERFLOW=5. Generates the complementary rollback diff and an on-call emergency cheat sheet with verification command: python target_system/demo_app/load_simulator.py --users 10.



Agent 5 evaluates the plan against deterministic risk policies. Because blast radius is elevated (100%), Agent 5 enforces Human-in-the-Loop approval gate while arming surrogate failover watchdog.



Upon HITL authorization, Agent 4 creates an atomic file backup in .umbrella_backups/, applies the patch to config.py, refreshes in-memory SQLAlchemy engine pools, and runs regression tests. Verification passes with 100% success (0% failure) in 3,017 ms.



Agent 6 logs the experience in evolution_memory.json, calculates RL reward (+12.5), and updates Q(s, a)=7.648. Under the 24-Hour observation window, it proactively formulates EVO-PROP-24H proposing POOL_SIZE=20 to permanently absorb future traffic growth.

EXPERIMENTAL VERIFICATION, PROVENANCE & ACADEMIC INTEGRITY

Automated test suite validation, open-source dataset lineage, and reproducibility declaration.

